

医小管 v2

大赛答辩提问准备稿

机器人与人工智能大赛 · 评委问答环节

AI 首答 + 人工接管 + 知识库自增长

已嵌入中文字体，适合跨阅读器查看

医小管 v2 大赛答辩准备稿

面向“机器人与人工智能大赛”评委提问环节。

目标：把项目讲成一个有真实场景、有工程闭环、有 AI 价值的校园智能服务系统，而不是一个普通聊天机器人。

1. 项目一句话

医小管 v2 是面向高校校园服务的 AI 助手系统，通过“AI 首答 + 人工接管 + 知识库自增长”闭环，把学生高频咨询、教师答复和知识库运营连接起来。

更口语一点：

我们不是单纯做一个问答机器人，而是做了一个校园服务闭环。学生先由 AI 快速回答，AI 解决不了时可以转人工老师，老师处理后的高频问题还能反哺知识库，让系统越用越懂学校。

2. 30 秒开场稿

各位老师好，我们的项目叫医小管 v2，定位是校园 AI 智能服务系统。它主要解决学生办事入口分散、辅导员重复答疑压力大、知识更新难的问题。

系统由学生端、教师端和 FastAPI 网关组成，AI 引擎采用自托管 Dify Chatflow，结合学校知识库做 RAG 检索回答。我们的核心创新是把问答做成闭环：AI 能答就流式回答，答不了就转人工，老师回复后系统统计高频未解答问题，再通过 AI 润色和审核发布进入知识库。这样既提升学生体验，也让学校知识库可以持续进化。

3. 技术架构速记

mermaid 片段

```
graph LR
    Student["UniApp/Vue3"] --> Gateway["FastAPI Gateway"]
    Teacher["UniApp/Vue3"] --> Gateway
    Gateway --> PG["PostgreSQL"]
    Gateway --> Redis["Redis"]
    Gateway --> Dify["Dify Chatflow / RAG"]
    Gateway --> Realtime["WebSocket / Centrifugo"]
    Realtime --> Student
    Realtime --> Teacher
```

可以这样讲：

- 前端：学生端负责提问、查看历史、转人工、校园服务入口；教师端负责工作台、接单回复、知识库补充和数据看板。
- 后端：FastAPI 网关统一处理认证、会话、状态机、消息持久化、Dify 调用、知识库运营和实时推送。
- AI：Dify Chatflow 负责意图分类和 RAG 回答，模型使用 qwen-plus，知识检索走学校知识库。
- 数据：PostgreSQL 存用户、会话、消息、知识条目、统计数据；Redis 支撑限流和缓存类能力。

- 实时：AI 流式回答用 SSE；老师接单、消息、状态变化用 WebSocket/Centrifugo 推送。

4. 最核心业务闭环

闭环 A：学生问答闭环

mermaid 片段

```
sequenceDiagram
    participant S as 
    participant G as Gateway
    participant D as Dify
    participant T as 
    S->>G: 
    G->>G: /
    alt AI 
        G->>D: Chatflow + RAG
        D-->>G: SSE token + message_end
        G-->>S: 
        G->>G: chat_analytics
    else 
        G-->>T: 
        G-->>S: JSON 
    end
```

闭环 B：AI + 人工协作闭环

会话状态机：

text 片段

```

ai_serving -> pending_teacher -> teacher_serving -> resolved -> ai_serving
      \          \          \          \
      \          \          \          \-> closed
      \          \          \          \
      \          \          \          \-> closed
      \          \          \          \
      \          \          \          \-> ai_serving(timeout)
      \          \          \          \
      \          \          \          \-> closed
      \          \          \          \
      \          \          \          \-> closed

```

讲法：

AI 默认接待学生。如果学生主动转人工，状态变成 pending_teacher，并通知同学院在线教师；教师接单后进入 teacher_serving，此时 AI 暂停，不会抢答；教师标记解决后进入 resolved，学生下一次继续提问时才恢复 AI 服务。这个状态机保证了 AI 和老师不会同时抢话，也方便审计整个服务过程。

闭环 C：知识库自增长闭环

text 片段

[illegible]

这条是答辩重点：

普通问答系统的知识库是静态的，我们的知识库会根据学生真实问题持续增长。未解答问题不是被丢掉，而是进入教师运营台，老师答完后经过 AI 润色和权限审核再入库。

5. 项目亮点

1. 不是通用聊天，而是校园业务系统

- 有学生、教师、管理员角色。
- 有会话、工单、状态机、知识库、数据看板。
- 回答围绕校园办事流程和学校知识库。

2. AI 与人工协作清晰

- AI 先答，提高 7x24 响应能力。
- 低置信或学生需要时转人工。
- 教师接管期间 AI 暂停，避免重复或冲突回答。

3. 知识库能自增长

- chat_analytics 记录问题、RAG 分数、命中文档、是否回答。
- unanswered_questions 聚合高频未覆盖问题。
- 教师答复后，系统自动润色、按作用域发布或审核。

4. 实时体验完整

- AI 回答用 SSE 实现逐字/逐段流式输出。
- 工单通知、老师接入、老师回复、状态变化用 WebSocket/Centrifugo 推送。

5. 权限边界明确

- 学生只能看自己的会话。
- 教师只能接本学院待处理或自己已接的会话。
- 全校知识发布需要管理员审核。
- Centrifugo 订阅代理限制用户不能订阅别人的频道。

6. 可运营、可观测

- 看板统计总提问、AI 解决率、平均响应时间、未解答 Top、学院分布、时段热力、Token 成本和延迟。
- 这些指标能反映 AI 效果和老师工作压力。

6. 评委高频问题与推荐回答

Q1：你们为什么要做这个项目？

推荐答法：

高校学生遇到问题时，信息往往分散在官网、服务大厅、通知、辅导员群里。对学生来说找入口成本高，对老师来说重复答疑压力大。医小管 v2 通过 AI 把常见问题先自动解答，把复杂问题转给老师，并把老师处理过的问题沉淀为知识库，解决的是“学生找不到、老师答不完、知识沉不下”的问题。

Q2：和普通 ChatGPT 或校园 FAQ 有什么区别？

推荐答法：

第一，它接入的是学校自己的知识库，不是泛泛聊天；第二，它有学生、教师、管理员角色和权限控制；第三，它支持转人工和实时接单，不是答完就结束；第四，它会统计 AI 未解决的问题，再让教师补知识，形成知识库自增长闭环。所以它更像一个校园智能服务平台，而不是单点问答工具。

Q3：AI 回答怎么避免胡编乱造？

推荐答法：

我们主要从三层控制。第一，Dify Chatflow 的 RAG prompt 要求基于检索资料回答，并禁止编造 URL 和电话。第二，Gateway 会解析 Dify 的 message_end metadata，提取 RAG 分数和引用来源，低置信答案会进入未解答统计。第三，涉及全校范围的新知识不能教师直接发布，需要管理员审核。也就是说，我们不只靠 prompt，而是用检索、评分、人工审核共同降低幻觉风险。

Q4：为什么选择 Dify？

推荐答法：

Dify 的优势是适合快速搭建和迭代 Chatflow。我们可以把意图分类、知识检索、RAG 回答和转人工提示配置成可视化流程，同时自托管部署，数据边界更可控。我们没有把业务逻辑全部放在 Dify 里，而是让 FastAPI Gateway 负责认证、状态机、权限、消息落库和知识库运营，Dify 专注 AI 编排，这样职责更清楚。

Q5：Dify Chatflow 具体怎么设计？

推荐答法：

入口先做意图分类，主要分为 greeting、chitchat、kb_query、transfer。问候和闲聊走轻量 LLM 回复；校园事务问题进入知识检索，再基于 RAG 结果回答；明确要求转人工时返回转人工提示。Gateway 调用 Dify 时会传入学生学院、校区、班级等 inputs，后续可以做学院个性化回答。

Q6：为什么既用 SSE 又用 WebSocket？

推荐答法：

两者解决的问题不一样。AI 回复是服务端连续往学生端推 token，用 SSE 更简单稳定；而师生会话状态、老师接单、老师回复、工单通知是多人实时协同，需要房间广播和用户定向推送，所以使用 WebSocket/Centrifugo。简单说：AI 流式输出用 SSE，实时协作用 WebSocket。

Q7：教师接入后 AI 还会不会继续回答？

推荐答法：

不会。会话进入 `teacher_serving` 后，学生发消息只会写库并实时广播给老师，Gateway 不再调用 Dify。老师点击解决后，状态变为 `resolved`；只有当学生继续提问时，系统才 `reactivate` 回 `ai_serving`。这能避免 AI 和老师同时回复造成体验混乱。

Q8：如果 AI 答不上怎么办？

推荐答法：

答不上不是终点，而是进入运营闭环。学生端会出现转人工入口；Gateway 会把未命中或低置信问题记录到 `chat_analytics` 和 `unanswered_questions`；教师端可以看到高频待补问题，老师补充答案后系统用 AI 润色，按班级、学院或全校范围发布到知识库。

Q9：知识库更新怎么保证质量？

推荐答法：

我们按作用域控制质量和效率。班级或学院范围的问题由对应教师直接发布，降低运营成本；全校范围的问题进入管理员审核，避免错误知识扩散。后台也会记录提交人、审核状态、Dify 文档 ID、发布时间等字段，方便追溯。

Q10：你们的创新点在哪里？

推荐答法：

我们的创新主要有三点：第一，AI 问答和人工教师不是割裂的，而是通过状态机形成可控协作；第二，知识库不是静态导入，而是基于学生真实问题持续自增长；第三，系统不仅面向学生，还给教师提供数据看板和知识运营工具，形成“服务、运营、改进”的闭环。

Q11：数据安全和权限怎么做？

推荐答法：

后端使用 JWT 认证，用户分为 `student`、`teacher`、`admin`。学生只能访问自己的会话；教师只能看到本学院待接单和自己处理中的会话；管理员可以做系统管理。实时通信也做了订阅校验，用户不能订阅别人的个人频道或无权访问的会话频道。另外 API Key、数据库密码等通过环境变量和 `gitignore` 管理，不放进公开代码。

Q12：系统能支撑并发吗？

推荐答法：

架构上 Gateway 使用 FastAPI async，Dify 调用和数据库访问都是异步路径；实时推送可以由 Centrifugo 承担连接压力；会话、消息、统计表都建了索引。当前是比赛和试点规模，后续如果并发上来，可以继续做网关多副本、连接池复用、Redis 缓存和 Dify 服务扩容。

Q13：怎么评估 AI 效果？

推荐答法：

我们不只看主观体验，还记录客观指标。chat_analytics 会记录用户问题、RAG 分数、命中文档、是否回答、Token、费用和延迟。教师端看板可以展示 AI 解决率、命中率分布、高频未解答问题、学院分布和时段热力。这些指标可以指导知识库补充和模型优化。

Q14：如果学校原有系统不能深度对接怎么办？

推荐答法：

我们把“深度办理”和“入口导航”分层处理。现阶段不强依赖教务系统接口，而是优先做问答、流程解释和服务入口跳转；后续如果拿到企业微信或服务大厅权限，可以把相同的意图识别能力扩展成企微机器人卡片或免登跳转。这样项目不会被第三方系统权限卡住。

Q15：目前还有哪些不足？

推荐答法：

第一，知识库质量需要持续清洗和扩充，尤其是学院差异化材料；第二，目前很多办事服务以入口跳转为主，深度办理还需要学校系统开放接口；第三，语义聚类目前偏轻量，未来可以引入 embedding 聚类提升同义问题合并效果；第四，生产环境还需要更完整的监控、审计和压力测试。

7. 被追问时的“硬依据”

主题	可以引用的实现依据
网关路由	<code>services/gateway/app/main.py</code> 挂载 auth、chat、conversations、knowledge、analytics、admin 等路由
AI 流式回答	<code>services/gateway/app/routers/chat.py</code> 使用 <code>StreamingResponse</code> ，调用 <code>dify_client.chat_stream</code>
Dify 客户端	<code>services/gateway/app/services/dify_client.py</code> 封装 <code>/chat-messages</code> SSE 和 dataset 文档创建
状态机	<code>services/gateway/app/services/state_machine.py</code> 定义 escalate/accept/resolve/reactivate/close
会话模型	<code>services/gateway/app/models/conversation.py</code> 定义会话状态和消息发送方
权限控制	<code>services/gateway/app/services/conversation_service.py</code> 控制学生、教师、管理员可见范围

主题	可以引用的实现依据
知识库闭环	<code>services/gateway/app/services/knowledge_service.py</code> 处理教师答复、AI 润色、发布和审核
数据看板	<code>services/gateway/app/routers/analytics.py</code> 汇总 AI 解决率、未解答 Top、成本和热力图
学生聊天页	<code>apps/student-app/src/pages/chat/index.vue</code> 实现 SSE 渲染、转人工、老师消息展示
教师接单页	<code>apps/teacher-app/src/pages/questions/detail.vue</code> 实现接单、回复、解决
教师知识库页	<code>apps/teacher-app/src/pages/knowledge/index.vue</code> 实现高频待补、提交答复、审核入口
Dify 流程	<code>deploy/dify/yixiaoguan-chatflow.yml</code> 记录意图分类、RAG、转人工分支

8. 演示顺序建议

1. 学生端首页：展示“智慧校园助理”和常用服务入口。
2. 学生提问：问一个校园问题，展示 AI 流式回答和参考资料。
3. 触发转人工：点击“转人工服务”，展示系统提示“已通知老师”。
4. 教师端工作台：展示待处理提问出现。
5. 教师接单回复：进入详情页，接单，发送回复并解决。
6. 学生端同步：展示老师回复和“问题已解决”状态。
7. 教师知识库：展示“高频待补问题”，提交教师答复，说明 AI 润色和审核发布。
8. 数据看板：展示 AI 解决率、未解答 Top、学院分布、Token 成本等运营指标。

9. 答辩时避免说太满

建议这样说：

- “当前我们已经实现了 AI 问答、转人工、教师回复、知识库补充和数据看板的核心链路。”
- “现阶段办事类服务以流程咨询和入口导航为主，深度办理需要学校开放接口或企业微信权限。”
- “RAG 幻觉不能完全消除，但我们通过检索约束、低置信统计、人工审核和知识库运营降低风险。”
- “当前适合比赛演示和试点验证，生产大规模推广还需要压力测试、监控告警和更多真实数据验证。”

不建议这样说：

- “AI 一定不会答错。”
- “已经完全接入所有学校系统。”
- “可以替代辅导员。”
- “知识库已经覆盖所有校园问题。”

10. 本地验证记录

当前本机路径：`/Users/easten/Documents/easten/yixiaoguan-v2`

已执行：

bash 片段

```
git clone https://github.com/eastenzero/yixiaoguan-v2.git
cd services/gateway
/Users/easten/.local/bin/python3.12 -m venv .venv312
.venv312/bin/pip install -r requirements.txt
JWT_SECRET=test-secret-for-local-pytest-please-ignore \
DATABASE_URL=postgresql+asyncpg://yxc:yxc_v2_pass@localhost:5432/yixiaoguan_v2 \
REDIS_URL=redis://localhost:6379/15 \
DIFY_API_URL=http://localhost:5001/v1 \
DIFY_API_KEY=test \
DIFY_GLOBAL_DATASET_ID=test \
.venv312/bin/python -m pytest -q
```

结果：

- 82 passed
- 5 failed
- 失败集中在 `tests/test_ai_pause_resume.py` 和 `tests/test_analytics_capture.py` 中直接调用被 `@limiter.limit` 包装的 `chat_send` 函数，未传 `Starlette Request` 对象；属于测试 harness 与限流装饰器的适配问题。
- Python 3.9 不适合本项目测试，因为代码和测试使用了 `datetime.UTC`、`list | None` 等新语法/特性；应使用 Python 3.11+，本机已用 Python 3.12 重跑。

仓库历史验证资料：

- `scripts/r07-e2e-smoke.py` 覆盖 `student escalate -> teacher accept -> teacher HTTP send -> AI pause/resume`。
- `docs/requirements/R07-■■■■■■■■■■.md` 记录远端端到端冲烟 38/38 PASS。
- `.tasks/hk-deploy/full-smoke-report-2026-05-03.md` 记录新域名复测中 Gateway health、教师工作台实时刷新、学生页老师接入/回复/解决状态推送等核心项已 PASS。

11. 考前速记卡

三个闭环：

- 服务闭环：学生提问 -> AI 回答 -> 转人工 -> 教师解决。
- 知识闭环：未解答统计 -> 教师补充 -> AI 润色 -> 发布入库。
- 运营闭环：数据看板 -> 找问题 -> 补知识/调流程 -> 提升 AI 解决率。

三个技术关键词：

- RAG：让回答基于学校知识库。
- 状态机：让 AI 和老师协作有边界。
- 实时通信：SSE 做 AI 流式输出，WebSocket/Centrifugo 做师生同步。

三个安全点：

- JWT + 角色权限。
- 教师按学院/接单范围访问。
- 全校知识管理员审核。

一句收尾：

医小管 v2 的目标不是让 AI 取代老师，而是让 AI 承担高频、标准化问题，把老师的精力释放给真正需要人工判断和关怀的学生问题。